

Snowflake Connector Proxy

Admin Runbook

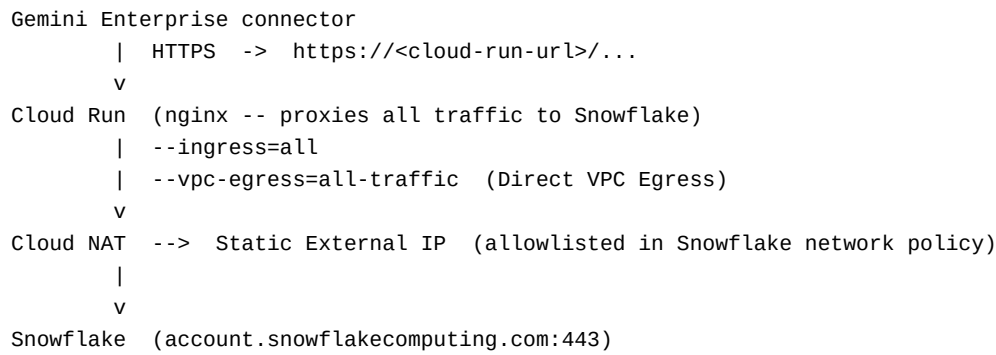
GCP Cloud Run · nginx · Cloud NAT · Gemini Enterprise

Last updated: May 08, 2026

1. Overview

ConnectorProxy is a GCP-hosted nginx reverse proxy that gives the Gemini Enterprise Snowflake connector a customer-controlled static egress IP. Gemini Enterprise uses a dynamic IP pool; Snowflake network policies require a fixed, allowlisted IP. All traffic is routed through Cloud Run whose egress is pinned to a static address via Cloud NAT.

2. Architecture



Direct VPC Egress connects Cloud Run instances directly to the VPC subnet without a Serverless VPC Access Connector. Combined with **--vpc-egress=all-traffic**, all outbound traffic from Cloud Run is routed through Cloud NAT, ensuring Snowflake always sees the static IP.

--ingress=all makes the Cloud Run URL publicly reachable — required for Gemini Enterprise to initiate the OAuth browser flow.

3. GCP Resources

The following resources are created during provisioning. Each provisioning step checks whether the resource already exists before creating it.

Resource	Purpose
Static external IP (regional)	Fixed egress address allowlisted in Snowflake network policy
Cloud Router	Required by Cloud NAT to manage routing within the VPC
Cloud NAT	Routes all Cloud Run egress through the static IP
Artifact Registry repository	Stores the nginx proxy Docker image
Cloud Run service	Runs nginx; Direct VPC Egress for static outbound IP

4. Prerequisites

Before starting, confirm the following are in place:

Requirement	Notes
gcloud CLI installed	Run: gcloud version

gcloud authenticated	Run: gcloud auth login
GCP project with billing enabled	Run: gcloud projects describe <PROJECT_ID>
Existing VPC network	The VPC that Cloud Run will use for Direct VPC Egress
Subnet within that VPC	Used by Cloud Run instances; must have sufficient IP space
Snowflake account hostname	Format: <account>.snowflakecomputing.com

Decide values for each of the following before starting. These are referenced in every provisioning step.

Variable	Description	Your value
PROJECT_ID	GCP project ID	
REGION	GCP region (e.g. us-central1)	
VPC_NETWORK	VPC network name	
VPC_SUBNET	Subnet name for Cloud Run Direct VPC Egress	
SNOWFLAKE_HOST	Snowflake hostname (e.g. xy12345.snowflakecomputing.com)	
SNOWFLAKE_PORT	Snowflake port	443
NAT_IP_NAME	Name to give the reserved static egress IP	
NAT_ROUTER_NAME	Name to give the Cloud Router	
NAT_GATEWAY_NAME	Name to give the Cloud NAT gateway	
CLOUD_RUN_SERVICE_NAME	Name to give the Cloud Run service	
AR_REPO	Name to give the Artifact Registry repository	

5. Provisioning

Follow each step in order. Replace all placeholder values (shown as `${VARIABLE_NAME}`) with your chosen values from Section 4. Each step includes a command to verify whether the resource already exists — only run the creation command if it does not.

Step 1 — Enable required GCP APIs

The following APIs must be enabled on the project. Running this command against already-enabled APIs is safe and has no effect.

```
gcloud services enable \
  compute.googleapis.com \
  run.googleapis.com \
  artifactregistry.googleapis.com \
  cloudbuild.googleapis.com \
  --project=${PROJECT_ID}
```

Confirm all APIs are enabled:

```
gcloud services list --enabled \
  --filter='name:(compute OR run OR artifactregistry OR cloudbuild)' \
  --project=${PROJECT_ID}
```

Step 2 — Reserve a static external IP address for Cloud NAT egress

This regional IP address is the fixed egress address that Snowflake will see for all traffic originating from the proxy. Reserve it once and record the address — you will add it to the Snowflake network policy in Section 7.

Check whether the address already exists:

```
gcloud compute addresses describe ${NAT_IP_NAME} \
  --region=${REGION} \
  --project=${PROJECT_ID}
```

If it does not exist, create it:

```
gcloud compute addresses create ${NAT_IP_NAME} \
  --region=${REGION} \
  --project=${PROJECT_ID} \
  --network-tier=PREMIUM
```

Retrieve and record the assigned IP address:

```
gcloud compute addresses describe ${NAT_IP_NAME} \
  --region=${REGION} \
  --project=${PROJECT_ID} \
  --format="value(address)"
```

Record this IP address — it is needed for the Snowflake network policy in Section 7.

Step 3 — Create a Cloud Router

Cloud NAT requires a Cloud Router in the same region and VPC. The router manages NAT configuration.

Check whether a router already exists in this region and VPC:

```
gcloud compute routers list \
  --region=${REGION} \
  --filter='network~${VPC_NETWORK}' \
  --project=${PROJECT_ID}
```

If a suitable router already exists, record its name and use it as `${NAT_ROUTER_NAME}` in subsequent steps. Otherwise, create a new one:

```
gcloud compute routers create ${NAT_ROUTER_NAME} \
  --network=${VPC_NETWORK} \
  --region=${REGION} \
  --project=${PROJECT_ID}
```

Step 4 — Create a Cloud NAT gateway

Cloud NAT translates outbound connections from the Cloud Run subnet, replacing the source IP with the static address reserved in Step 2. The gateway is scoped to `${VPC_SUBNET}` — the subnet used by Cloud Run Direct VPC Egress.

Check whether a NAT gateway already exists on the router:

```
gcloud compute routers nats describe ${NAT_GATEWAY_NAME} \
  --router=${NAT_ROUTER_NAME} \
  --region=${REGION} \
  --project=${PROJECT_ID}
```

If it does not exist, create it:

```
gcloud compute routers nats create ${NAT_GATEWAY_NAME} \
  --router=${NAT_ROUTER_NAME} \
  --region=${REGION} \
  --project=${PROJECT_ID} \
  --nat-external-ip-pool=${NAT_IP_NAME} \
  --nat-custom-subnet-ip-ranges=${VPC_SUBNET}
```

--nat-custom-subnet-ip-ranges scopes the NAT gateway to the Cloud Run subnet only, so only traffic from Cloud Run instances is routed through the static IP.

Step 5 — Create an Artifact Registry Docker repository

The Docker image built in Step 6 is stored here. Cloud Build must be granted write access to push the image.

Check whether the repository already exists:

```
gcloud artifacts repositories describe ${AR_REPO} \
  --location=${REGION} \
  --project=${PROJECT_ID}
```

If it does not exist, create it:

```
gcloud artifacts repositories create ${AR_REPO} \
  --repository-format=docker \
  --location=${REGION} \
  --project=${PROJECT_ID}
```

Grant the Cloud Build service account permission to push images. First retrieve the project number:

```
gcloud projects describe ${PROJECT_ID} \
  --format="value(projectNumber)"
```

Then grant the role (replace \${PROJECT_NUMBER} with the value returned above):

```
gcloud artifacts repositories add-iam-policy-binding ${AR_REPO} \
  --location=${REGION} \
  --project=${PROJECT_ID} \
  --member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com" \
  --role=roles/artifactregistry.writer \
  --condition=None
```

Step 6 — Create the proxy Docker image files and build the image

The proxy container is an `nginx:alpine` image configured to forward all requests to Snowflake over TLS. Create the following three files in a directory named `proxy/`, then build and push the image using Cloud Build. No local Docker installation is required.

File 1 of 3 — `proxy/Dockerfile`

Uses `nginx:alpine` as the base. `ca-certificates` enables `nginx` to verify Snowflake's TLS certificate. `gettext` provides `envsubst`, which substitutes environment variables into the `nginx` config template at startup.

```
FROM nginx:alpine

RUN apk add --no-cache ca-certificates gettext

COPY nginx.conf.template /etc/nginx/nginx.conf.template
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

EXPOSE 8080

ENTRYPOINT ["/entrypoint.sh"]
```

File 2 of 3 — `proxy/entrypoint.sh`

Runs when the container starts. Reads the two environment variables set by Cloud Run (`SNOWFLAKE_HOST` and `SNOWFLAKE_PORT`), substitutes them into the `nginx` config template to produce the final config, validates it with `nginx -t`, then starts `nginx` in the foreground.

```
#!/bin/sh
set -e

envsubst '${SNOWFLAKE_HOST} ${SNOWFLAKE_PORT}' \
  < /etc/nginx/nginx.conf.template \
  > /etc/nginx/nginx.conf

nginx -t
exec nginx -g 'daemon off;'
```

File 3 of 3 — `proxy/nginx.conf.template`

`nginx` reverse proxy configuration. All incoming requests on port 8080 are forwarded to Snowflake over TLS. Buffering is disabled so Server-Sent Events (used by the Gemini MCP connection) are streamed immediately. The placeholders `${SNOWFLAKE_HOST}` and `${SNOWFLAKE_PORT}` are replaced by `entrypoint.sh` at startup.

```
events {
    worker_connections 1024;
}

http {
    access_log /dev/stdout;
    error_log /dev/stderr;

    server {
        listen 8080;
        server_name _;
```

```

# Disable buffering for MCP Server-Sent Events
proxy_buffering      off;
proxy_cache          off;
chunked_transfer_encoding on;

location / {
    proxy_pass          https://${SNOWFLAKE_HOST}:${SNOWFLAKE_PORT};
    proxy_http_version  1.1;

    proxy_set_header    Host      "${SNOWFLAKE_HOST}";
    proxy_set_header    Connection "";

    # Verify Snowflake TLS certificate
    proxy_ssl_server_name on;
    proxy_ssl_name       "${SNOWFLAKE_HOST}";
    proxy_ssl_verify      on;
    proxy_ssl_trusted_certificate /etc/ssl/certs/ca-certificates.crt;

    proxy_pass_request_headers on;

    proxy_connect_timeout 10s;
    proxy_read_timeout    3600s;
    proxy_send_timeout     3600s;
}
}

```

Build and push the image using Cloud Build:

Run the following command from the directory that contains the `proxy/` folder. Cloud Build uploads the files, builds the image remotely, and pushes it to Artifact Registry.

```

gcloud builds submit proxy/ \
  --tag=${REGION}-docker.pkg.dev/${PROJECT_ID}/${AR_REPO}/proxy:latest \
  --project=${PROJECT_ID}

```

Confirm the image was pushed successfully:

```

gcloud artifacts docker images list \
  ${REGION}-docker.pkg.dev/${PROJECT_ID}/${AR_REPO} \
  --project=${PROJECT_ID}

```

Step 7 — Deploy the Cloud Run service

Deploys the nginx proxy as a Cloud Run service using Direct VPC Egress. Cloud Run instances connect directly to the VPC subnet — no Serverless VPC Access Connector is required. If the service already exists this command updates it. Key flags:

Flag	Purpose
<code>--network</code>	VPC network for Direct VPC Egress
<code>--subnet</code>	Subnet within the VPC that Cloud Run instances use
<code>--vpc-egress=all-traffic</code>	Routes all outbound traffic through the VPC and Cloud NAT
<code>--set-env-vars</code>	Passes Snowflake host and port into the container at startup

--ingress=all	Makes the Cloud Run URL publicly reachable (required for Gemini Enterprise OAuth)
--allow-unauthenticated	Allows Gemini Enterprise to call the service without GCP credentials
--max-instances=3	Limits concurrent instances; adjust based on expected traffic

```
gcloud run deploy ${CLOUD_RUN_SERVICE_NAME} \
  --image=${REGION}-docker.pkg.dev/${PROJECT_ID}/${AR_REPO}/proxy:latest \
  --region=${REGION} \
  --project=${PROJECT_ID} \
  --network=${VPC_NETWORK} \
  --subnet=${VPC_SUBNET} \
  --vpc-egress=all-traffic \
  --set-env-vars="SNOWFLAKE_HOST=${SNOWFLAKE_HOST}, SNOWFLAKE_PORT=${SNOWFLAKE_PORT}" \
  --port=8080 \
  --cpu=1 \
  --memory=512Mi \
  --max-instances=3 \
  --ingress=all \
  --allow-unauthenticated
```

Retrieve the Cloud Run service URL once deployment completes:

```
gcloud run services describe ${CLOUD_RUN_SERVICE_NAME} \
  --region=${REGION} \
  --project=${PROJECT_ID} \
  --format="value(status.url)"
```

Record this URL — it is used for all three Gemini Enterprise connector fields in Section 6.

6. Gemini Enterprise Connector Configuration

In the Gemini Enterprise console, open the Snowflake connector configuration and enter the following values. Use the Cloud Run URL retrieved at the end of Step 7.

Field	Value
MCP URL	https://<cloud-run-url>/api/v2/databases/<db>/schemas/<schema>/mcp-servers/<server>
Authorization URL	https://<cloud-run-url>/oauth/authorize
Token URL	https://<cloud-run-url>/oauth/token-request
Client ID	OAuth client ID from the Snowflake security integration
Client Secret	OAuth client secret from the Snowflake security integration
Scopes	session:role:<ROLE_NAME>

Critical: All three URLs must use the exact Cloud Run URL with no typos. A wrong Token URL causes a silent failure — the browser OAuth login succeeds but the token exchange never reaches the proxy, resulting in "Failed to obtain refresh token".

The Token URL must end in /oauth/token-request, not /oauth/authorize. These are two different Snowflake endpoints.

7. Snowflake Configuration

Network policy — allowlist the static egress IP

Add the static IP reserved in Step 2 to the Snowflake network policy. All traffic from the proxy to Snowflake originates from this IP. Run the following in Snowsight or via SnowSQL:

```
ALTER NETWORK POLICY <your_policy_name>
  ADD ALLOWED_IP_LIST = ('<static_ip_from_step_2>/32');
```

If no network policy exists yet, create one:

```
CREATE NETWORK POLICY <policy_name>
  ALLOWED_IP_LIST = ('<static_ip_from_step_2>/32');

ALTER ACCOUNT SET NETWORK_POLICY = <policy_name>;
```

OAuth security integration

Create a Snowflake OAuth security integration for the Gemini Enterprise connector. The redirect URI must be the Google OAuth redirect endpoint. `OAUTH_ISSUE_REFRESH_TOKENS = TRUE` is required — without it the connector cannot maintain long-lived access and will fail after the initial access token expires.

```
CREATE SECURITY INTEGRATION <integration_name>
  TYPE = OAUTH
  ENABLED = TRUE
  OAUTH_CLIENT = CUSTOM
  OAUTH_CLIENT_TYPE = CONFIDENTIAL
  OAUTH_REDIRECT_URI = 'https://vertexaisearch.cloud.google.com/oauth-redirect'
  OAUTH_ISSUE_REFRESH_TOKENS = TRUE
  OAUTH_REFRESH_TOKEN_VALIDITY = 7776000; -- 90 days
```

Retrieve the client ID and secret to enter in the Gemini connector:

```
SELECT SYSTEM$SHOW_OAUTH_CLIENT_SECRETS('<integration_name>');
```

8. Verification

Confirm the proxy forwards requests to Snowflake:

Any response from Snowflake — including 401 Unauthorized — confirms the proxy is running and forwarding traffic correctly.

```
curl -si https://<cloud-run-url>/api/v2/ping | head -10
```

Confirm Snowflake sees the static egress IP:

Run the following in Snowsight after triggering any request through the proxy. The `client_net_address` column must show the static IP reserved in Step 2.

```
SELECT client_net_address, user_name, start_time
FROM TABLE(information_schema.query_history(
  daterange_start => dateadd('hour', -1, current_timestamp())))
ORDER BY start_time DESC
LIMIT 5;
```

Check Cloud Run request logs:

A healthy OAuth flow produces four requests in sequence. If `POST /oauth/token-request` is absent, the Token URL in the Gemini connector is misconfigured.

```

gcloud logging read \
  "resource.type=cloud_run_revision \
    AND resource.labels.service_name=${CLOUD_RUN_SERVICE_NAME}" \
  --project=${PROJECT_ID} \
  --limit=50 \
  --freshness=10m \
  --format="value(timestamp, httpRequest.requestMethod, \
    httpRequest.requestUrl, httpRequest.status)"

```

Expected sequence for a successful OAuth flow:

Request	Expected status
GET /oauth/authorize	200
POST /session/authenticate-request	200
POST /oauth/authorization-request	200
POST /oauth/token-request	200

9. Security Model

Layer	How it protects
Snowflake OAuth	Valid client credentials (client ID + secret) required to initiate any flow
Snowflake network policy	Only the static egress IP is allowlisted — all other source IPs are rejected by Snowflake